

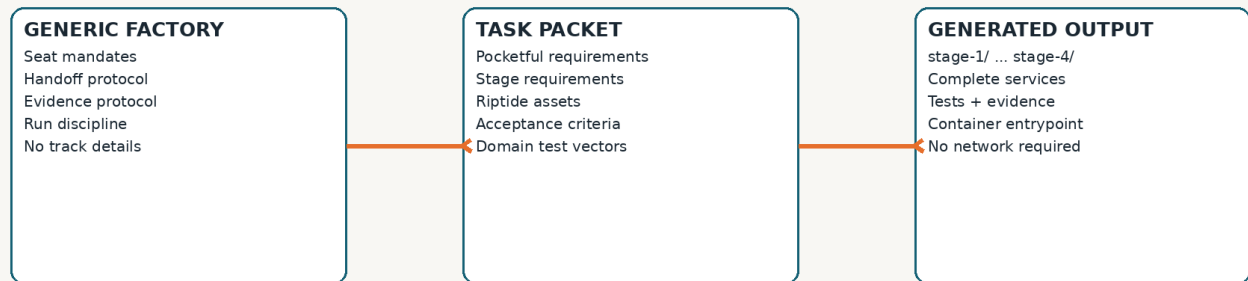
DARK FACTORY

POCKETFUL + RIPTIDE

MASTER ENGINEERING & SUBMISSION SPECIFICATION

Rule-correct rebuild for BAND Desktop — 26 September 2026

Submission Architecture: Generic Factory, Specific Task



Hard rule: track-specific detail belongs in the task packet, never in a seat mandate.

JUDGED EVIDENCE

BAND room export + recording | mandate files | factory description | public repo | completed stage folders

Purpose of this rebuild

Replace the earlier product-centric plan with a competition-correct architecture in which the standing BAND seat mandates are fully generic, Pocketful/Riptide exists only in the run task packet, each completed stage is a complete buildable service, the service survives a clean no-outbound-network environment, and the submission contains the factory, the room/run evidence, and the generated result.

Prepared for a three-human-contributor team. Agent-seat responsibilities and human responsibilities are deliberately separated throughout this document.

1. Document Control and Rule Reset

This specification supersedes the previous Pocketful and Riptide integration documents for the hackathon submission. The previous engineering work remains useful as product knowledge, test knowledge, and asset preparation; however, it must no longer be allowed to leak into the standing agent mandates. The screenshot supplied by the team introduces explicit disqualification conditions that require a structural reset rather than a small edit.

Primary reset

The factory is the reusable artifact. Pocketful is one task run through that factory. Riptide is a task-specific product layer. A mandate must still make sense if the same seat were handed an unrelated software problem tomorrow.

Item	Current interpretation	Engineering consequence
BAND seats	At least three distinct coding-agent seats, each with a mandate file.	Use four generic seats for clearer separation: Planner/Architect, Builder/Integrator, Breaker/Test Engineer, Release Verifier.
Mandates	Must be generic; track-specific details in a mandate are disqualifying.	No Pocketful, Venmo, payments, money, Riptide, endpoint path, field name, schema name, test value, or expected error code in mandate files.
Task detail	Track detail belongs in the task posted to the room.	Create a task packet containing official stage requirements, Pocketful invariants, Riptide assets, and product acceptance criteria.
Repository	One public GitHub repository cloneable without BAND membership.	Put mandates, factory description, BAND room export, and each completed self-contained stage in the public repo.
Stages	One folder per completed stage, each a complete buildable service.	Snapshot only independently accepted stages. Never make stage-1 depend on files inside stage-4.
Video	Must include recording of the BAND room that generated the solution.	Capture the actual room/run; polished demo-only video is unsafe.
Service	Must build and serve from clean container with no outbound network.	Bundle assets locally; remove runtime external APIs, CDNs, model calls, hosted databases, analytics, and hidden package fetches.

Rule basis: team-supplied checklist screenshot; current official event pages describe a BAND software factory, four graded stages, and submission of the factory, the run, and the result. Current LabLab live dashboard lists submissions closing 06:59 UTC on 6 Oct 2026 (11:59 PM Pacific on 5 Oct).

2. Competition Source of Truth and Change-Control Policy

This document distinguishes verified competition constraints from our own implementation choices. That distinction is critical because the stage specification and harness limits are published at kickoff. We should not convert our guesses into factory standing instructions. The run task packet is the correct place to insert stage-specific requirements once the official specification is available.

2.1 Authority order

1. Official kickoff specification and any organizer-issued corrections or clarifications.
2. BAND and LabLab submission checklist for mandatory artifacts and disqualifiers.
3. Track specification for the selected Pocketful stage.
4. This engineering specification for internal implementation decisions and optional Riptide product enhancements.
5. Existing Hidden Canopy repositories as design/test references only; they do not override the challenge spec.

2.2 Stage requirements are deliberately not invented here

The event marketing confirms four graded stages but, at the time of this rebuild, the exact stage-by-stage contracts are not exposed in the public pages we could verify. The repository design therefore supports all four stage snapshots without hard-coding speculative stage content. When kickoff publishes the stage specification, Contributor B updates only the task packet and acceptance matrix; the generic mandates remain unchanged.

Do not “pre-bake” the challenge into the factory

If a later kickoff requirement uses a field name, endpoint, error code, dataset, fixed seed, or stage-specific behavioral detail, put it in the run task packet or stage acceptance file. Never retrofit it into a standing mandate.

3. What We Are Actually Submitting

The submission has three distinct layers, and judges should be able to identify them immediately. The first is the factory: the reusable BAND seat configuration, generic mandates, handoff discipline, and verification flow. The second is the run: the task posted to the room, the actual room transcript/export, human intervention log, and evidence that the agents planned, implemented, tested, rejected, corrected, and accepted work. The third is the result: the completed stage service folders, each independently buildable and runnable.

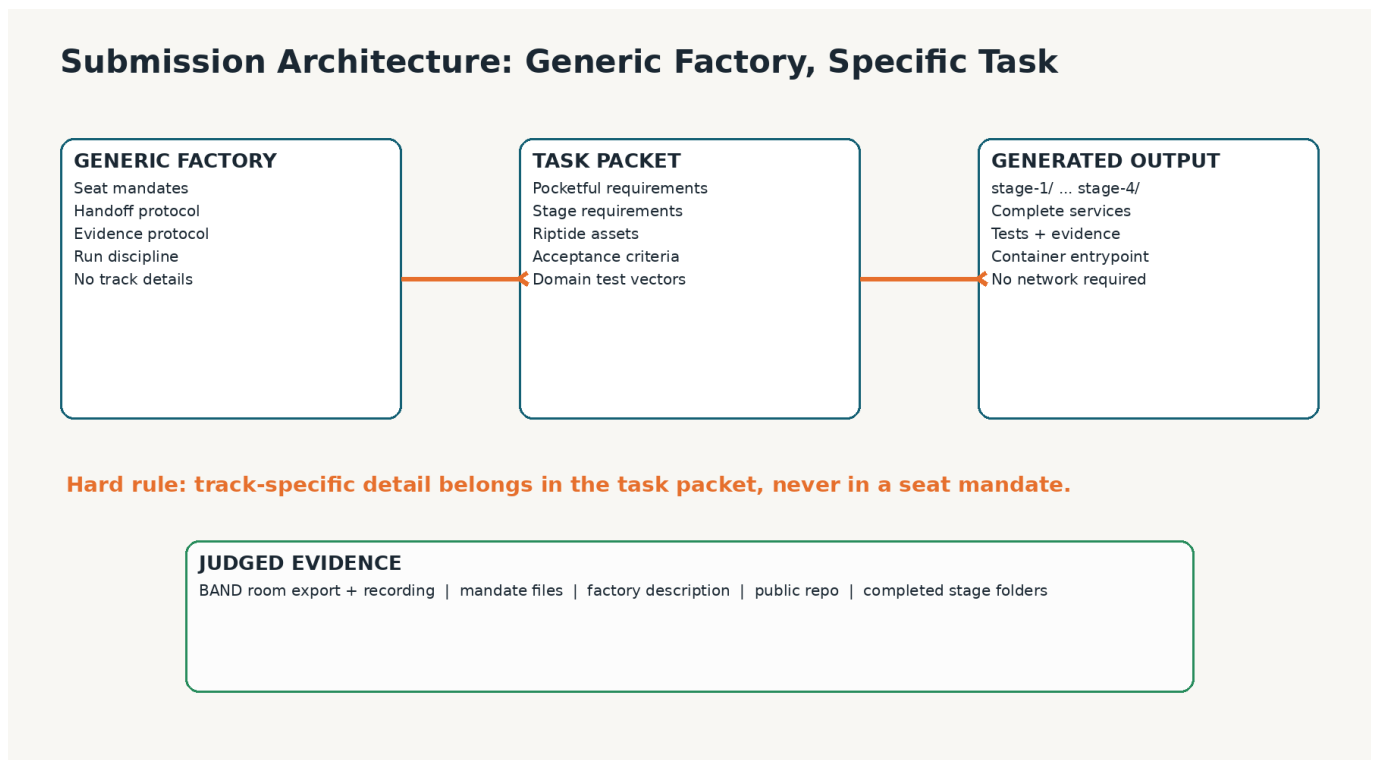


Figure 1. The compliance boundary that drives the entire rebuild.

3.1 Factory artifact

- Generic mandate files for every seat.
- Factory description explaining planning, implementation, independent breakage, release verification, handoffs, and rejection loops.
- Generic evidence schemas and run manifest.
- BAND Desktop room export.

3.2 Run artifact

- Pocketful/Riptide task packet posted to the room.
- Task decomposition and handoff receipts.
- Actual build/test/rejection evidence.
- Human intervention log with timestamps and reasons.
- Stage acceptance verdicts.

3.3 Result artifact

- One self-contained folder per completed official stage.

- Container build/start instructions per stage.
- Tests and locally bundled static assets.
- Evidence index linking observable requirements to reproducible commands/results.

4. Public Repository Blueprint

The repository must be legible to a judge who has no BAND Desktop membership. The public repository is therefore both a build artifact and an audit surface. Every path should make its role obvious, and a fresh clone should contain everything required to understand the factory and build any submitted stage.

```

/
  README.md
  factory-description.md
  mandates/
    01-planner-architect.md
    02-builder-integrator.md
    03-breaker-test-engineer.md
    04-release-verifier.md
  band-room-export/
    <export supplied by BAND Desktop>
  task-packets/
    pocketful-run.md
  stage-1/
    <complete self-contained service>
  stage-2/
    <complete self-contained service if completed>
  stage-3/
    <complete self-contained service if completed>
  stage-4/
    <complete self-contained service if completed>
  submission/
    evidence-index.md
    intervention-log.md
    video-notes.md

```

4.1 Stage folder independence

A submitted stage must not be a patch, a diff, or a symbolic pointer to the newest code. It should build as if it were the only stage in the repository. Duplicating a modest amount of source across snapshots is preferable to creating fragile cross-stage dependencies that fail when a judge enters one folder and runs the documented command.

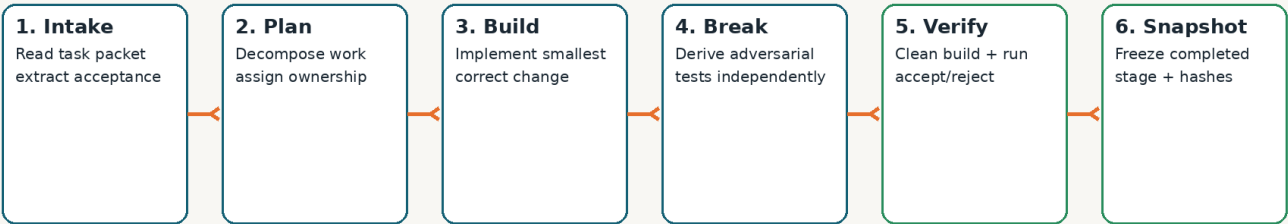
4.2 Public does not automatically mean open source

The checklist requires a public GitHub repository. That is not the same statement as an MIT or other open-source license requirement. Do not add an open-source license merely because the repository is public unless the organizers explicitly require one. Keep the character artwork and any non-code asset rights clearly stated in the repository.

5. Generic Agent-Seat Architecture

We use four seats rather than the minimum three because the extra separation makes the independence story easier to prove. All four are coding-capable seats. Their mandates describe how they work, how they hand off, and when they reject work; none names the Pocketful track, a financial domain, Riptide, or any challenge-specific detail.

Generic BAND Run Flow



No seat accepts its own work. Rejection returns evidence to a different seat for correction.

All product-specific instructions arrive with the task, so the same factory can build a different service unchanged.

Seat	Standing role	May author production code?	May accept own production code?
Planner / Architect	Extract requirements, identify risk, define interfaces, decompose work.	Yes, if explicitly assigned foundational work.	No.
Builder / Integrator	Implement assigned changes and produce evidence.	Yes.	No.
Breaker / Test Engineer	Derive independent adversarial tests from task requirements.	Test/harness code; not candidate production fixes while breaking.	No.
Release Verifier	Clean build/run and independent verdict.	Only if relinquishing verifier authority for that candidate.	No.

5.1 Why not name seats by domain

Names such as “Financial Ledger Engineer,” “Pocketful Backend,” “Riptide UX Agent,” or “Payments QA” would encode the challenge into the factory. Even apparently harmless standing instructions such as

“always verify debit and credit postings” are domain-specific. The mandates should instead say to identify and test invariants derived from the current task.

6. Mandate Compliance Audit

Before the BAND run begins, Contributor A should perform a literal-string audit over every mandate file. The goal is not merely to remove the word Pocketful; it is to remove hidden challenge fingerprints. A generic mandate should not know that a wallet exists, that money is measured in cents, that requests have a particular key, or that the track uses any specific interface.

Forbidden in standing mandate	Why	Where it belongs instead
Product or track name	Direct challenge specificity.	Task packet.
Domain nouns such as wallet, ledger, booking table, merchant	Pre-bakes one problem family.	Task packet or generated design.
Endpoint paths	Explicitly called out by organizers as non-generic detail.	Task packet / implementation.
Field names / schema names	Encodes the challenge data model.	Task packet / implementation.
Error codes or exact error strings	Encodes expected track behavior.	Acceptance specification.
Specific numeric thresholds/test values	Can reveal track fixture expectations.	Task acceptance tests.
Riptide states, sprites, prompt text	Product-specific UI behavior.	Pocketful/Riptide job packet.

6.1 Generic mandate self-test

For every sentence, ask: “Would this instruction still make sense if tomorrow the same factory were asked to build a document editor, image service, queue processor, or scheduling engine?” If the answer is no, remove it from the mandate and move it into the run task.

7. Handoff and Evidence Protocol

The factory should not communicate completion through conversational confidence. Each handoff is a small evidence-bearing object. This makes agent cooperation observable and gives the verifier a stable interface that is generic across domains.

7.1 Required handoff fields

Field	Purpose
task_id	Stable identifier for the unit of work.
producer_seat	Seat that generated the artifact.
consumer_seat	Seat responsible for the next action.
intent	What was supposed to change.
files_changed	Paths touched by the producer.
commands_run	Reproduction commands, not “I tested it.”
tests	Named tests with commands and pass/fail state.
evidence	Paths and hashes for logs/artifacts.
assumptions	Explicit uncertainty introduced by the producer.
unresolved_risks	Known gaps that the next seat must consider.
status	Ready, rejected, or blocked.

```
{
  "task_id": "T-017",
  "producer_seat": "builder",
  "consumer_seat": "breaker",
  "intent": "Implement assigned behavior",
  "files_changed": [...],
  "commands_run": [...],
  "tests": [{ "name": "...", "result": "pass" }],
  "evidence": [{ "path": "...", "sha256": "..."}],
  "assumptions": [],
  "unresolved_risks": [],
  "status": "ready_for_review"
}
```

8. Run Manifest and Intervention Log

A run manifest lets a judge distinguish the factory configuration from the challenge task and from later human activity. It also protects us from accidentally turning an agent-assisted coding session into an undocumented manual build.

8.1 Run manifest

- Run identifier and timestamps.
- SHA-256 of every mandate file at run start.
- SHA-256 of the task packet posted into the room.
- Model/runtime information for each seat where exportable.
- Stage candidate commit identifiers.
- Independent verdict per stage.
- Clean build/start command and result.
- Known limitations.

8.2 Human intervention log

Any human correction after the task enters the room should be recorded. The preferred pattern is to post a correction or clarification into the room and let a seat make the code change. If a human edits generated product code directly, record the exact files and reason; then re-run independent verification. A hidden manual patch undermines the factory narrative and makes the recorded room less representative of the result.

9. Four-Stage Snapshot Mechanism

The repository should contain only completed stages, and each stage should be independently buildable. The factory therefore needs a generic snapshot action that occurs after a verifier accepts a stage. The action copies or materializes the accepted candidate into the required stage folder, writes build/test instructions, and records the candidate commit and evidence hash.

9.1 Snapshot gate

6. Official stage acceptance criteria loaded from task packet.
7. Candidate produced by a builder seat.
8. Breaker executes independent probes.
9. Verifier rebuilds/starts candidate from clean state.
10. Verifier issues ACCEPT.
11. Snapshot generated and hashed.
12. Fresh clone test performed on snapshot folder.

9.2 Do not guess stage semantics

This document intentionally does not assert what Stage 1 through Stage 4 mean functionally. The kickoff specification must supply those details. Our machinery is stage-aware but domain-agnostic: it can snapshot any completed official stage without changing the standing mandates.

10. Clean-Container / No-Outbound Engineering Strategy

The service constraint should shape the implementation from the first line of code. A submission that works on a developer laptop but reaches package registries, CDNs, remote model APIs, hosted databases, analytics endpoints, or external fonts during the judged build/run is structurally unsafe.

Target posture

A judge can clone the repo, enter a stage folder, build the container in the supported environment, start the service with outbound networking unavailable, and exercise all required behavior with local state and local assets.

Dependency class	Default decision	Rationale
Runtime web framework	Avoid unless vendored/guaranteed.	Reduces package resolution risk.
Database	Local SQLite via Python stdlib is viable for a single-service challenge build.	Durable, transactional, offline, no separate server.
Frontend	Plain bundled HTML/CSS/JS.	No package-manager or CDN dependency required.
Fonts/icons	Local assets or system-safe fallbacks.	No external fetch.
Riptide imagery	Commit assets in service folder.	Works offline and makes asset provenance explicit.
LLM for Riptide runtime	Do not require.	Network/model availability would make judged behavior fragile and non-deterministic.
Analytics/telemetry	Local evidence only.	Avoid outbound calls and privacy noise.

11. Proposed Runtime Baseline

For the hackathon result, the most robust baseline is a small Python service using only the standard library where practical: ``http.server`` or a tiny custom request layer, ``sqlite3`` for durable transactional state, ``json`` for contracts, ``threading``/locks only where needed, and static frontend files served locally. This is not a claim that standard-library Python is the ideal production stack; it is a deliberate response to the clean-container/no-network constraint and the short build window.

11.1 Why SQLite can work here

SQLite serializes writes and provides durable transactions without a separate database process. The transfer path should begin a write transaction before checking mutable balances, enforce idempotency under a unique constraint, write the logical transaction and its postings atomically, and commit once. Busy timeouts and bounded retry behavior should be explicit. The service should expose enough concurrency in HTTP handling to allow the harness to race requests, while the database transaction boundary prevents double-spend.

11.2 Alternative

If the official harness explicitly supports multi-container services or a preinstalled PostgreSQL environment, PostgreSQL row locking may be preferable. The factory should discover that from the task/runtime context rather than having the mandate prescribe a database.

12. Pocketful Product Contract

Pocketful is the current task, not the factory. The product should be recognizable as a consumer wallet/payments application rather than an enterprise financial dashboard. The core surfaces are balance, send/request, activity, budgets, savings goals, safe-to-spend context, and Riptide as a decision companion.

12.1 Core user flows

- Open home and see authoritative current balance plus recent activity.
- Send value to another local user/account.
- Request value and resolve the request locally.
- Inspect transaction state: pending, completed, failed, or rejected.
- Create budgets/categories and view remaining amounts derived from completed transaction history.
- Create savings goals and show progress.
- Ask Riptide to intervene at configured levels when a purchase would conflict with user-defined financial priorities.

12.2 Non-goals for judged build

- No claim of real banking connectivity.
- No claim of live FedNow, card acquiring, or external settlement.
- No requirement for remote identity providers.
- No runtime dependency on a cloud model.
- No hidden privileged admin path that bypasses financial invariants.

13. Authoritative Money Model

The authoritative monetary representation should be integer minor units for USD-facing operations. The service should never derive posted money from binary floating-point arithmetic. String or integer request parsing is validated into an integer number of cents before any state transition.

Invariant M1 — conservation

For every completed internal transfer, the algebraic sum of all system postings created by that transfer is zero. Value may move between accounts; it may not appear or disappear inside the internal transfer boundary.

Invariant M2 — exactness

For every amount accepted by the USD contract, the posted amount is an exact integer number of cents. There is no rounding drift across repeated operations.

13.1 Funding boundary

If the demo needs seeded balances, represent funding as an explicit event from a designated system/external account rather than mutating user balances without accounting. This preserves global double-entry semantics while still allowing deterministic fixtures.

14. Ledger and Posting Model

The strongest internal model separates the logical transaction from the immutable postings that move value. A transfer object answers “what operation did the user request and what state is it in?” Postings answer “what value movement was committed?” A cached account balance may exist for performance, but it is a transactional projection that can be rebuilt from postings.

Entity	Key fields	Rules
account	id, owner_id, currency, status, cached_balance_minor	Balance changes only inside the same transaction that creates authoritative postings.
transfer	id, scoped_request_key, request_fingerprint, source, destination, amount_minor, state	Logical command; unique request key; immutable payload after admission.
posting	id, transfer_id, account_id, signed_amount_minor	Immutable after commit; sum per internal transfer = 0.
budget	id, owner_id, category, limit_minor, period	Projection policy; never writes ledger truth.
goal	id, owner_id, target_minor, current/projection metadata	Derived/product state; never creates money.
riptide_event	decision id, reason codes, severity, context hash, user outcome	Explains intervention; cannot mutate ledger directly.

15. Idempotency and Retry Semantics

A timeout is not proof that an operation failed. The logical request therefore needs a caller-provided idempotency key scoped to the appropriate actor or account context. Application-level “check then insert” is insufficient under simultaneous requests unless the database also enforces uniqueness.

15.1 Required behavior

13. First valid request with a new key attempts the operation.
14. Same key + same canonical payload returns the original logical result and creates no second financial effect.
15. Same key + different payload is a conflict, never a second operation.
16. Concurrent requests racing on the same key are serialized by the uniqueness boundary so only one winning logical transaction exists.
17. Restart/replay with an already committed key returns the persisted result.

15.2 Request fingerprint

Canonicalize the fields that define the logical financial instruction, then hash them. The persisted fingerprint lets the service distinguish a legitimate retry from a client accidentally reusing a key for a different transfer.

16. Concurrency and Double-Spend Defense

The critical adversarial case is two or more requests spending the same available funds at the same time. The earlier Hidden Canopy legacy FinTech pattern demonstrated why checking a balance before obtaining the write serialization boundary is unsafe. Pocketful should validate spendability only after the authoritative transaction lock is acquired.

16.1 SQLite strategy

Begin a write transaction that serializes the mutable account state; read the source and destination inside that transaction; validate status/currency/funds; insert the logical transaction/postings; update cached balances; commit. The losing concurrent request then observes the updated source state and fails or produces the required task-defined result. Keep the transaction small so serialization does not become unnecessary latency.

16.2 Deterministic lock ordering

If the chosen persistence layer exposes multiple row locks, acquire them in deterministic account-id order to reduce deadlock risk. This is an implementation tactic, not something that belongs in the generic mandate.

17. Transaction State Machine and Crash Consistency

State	Meaning	Permitted next states
received	Request parsed and canonicalized.	rejected, admitted
admitted	Idempotency and basic validation accepted.	committed, rejected
committed	All financial postings and balance projection committed atomically.	terminal
rejected	No financial effect occurred.	terminal

For the hackathon local service, avoid multi-step “debit then later credit” workflows that create crash windows. Both postings and both account projection changes belong in one local database transaction. If a future external rail is added, external dispatch needs its own durable-intent and exactly-once-at-the-boundary design, but that is outside the judged local service unless required by the official stage.

18. Reconciliation and Evidence of Truth

A reconciliation command or test should recompute each account's authoritative balance from postings and compare it with the cached balance. The service should treat a mismatch as an integrity failure rather than quietly correcting it without evidence. Reconciliation is both a product safeguard and a powerful hackathon proof because it demonstrates that displayed balances are not arbitrary mutable counters.

18.1 Suggested evidence

- Starting total system value.
- Ending total system value.
- Count of completed transfers.
- Count of rejected insufficient-fund attempts.
- Count of duplicate logical requests and number of actual financial effects.
- Number of accounts whose cached balance differs from recomputation.
- Number of unbalanced transfer posting groups.

19. Budgets, Goals, and Safe-to-Spend

Budgets and goals are projections downstream of authoritative transactions. They may influence Riptide prompts, but they must not mutate money. A user can change a budget limit without altering historical transactions. Safe-to-spend is likewise a derived advisory metric with a documented formula and timestamp.

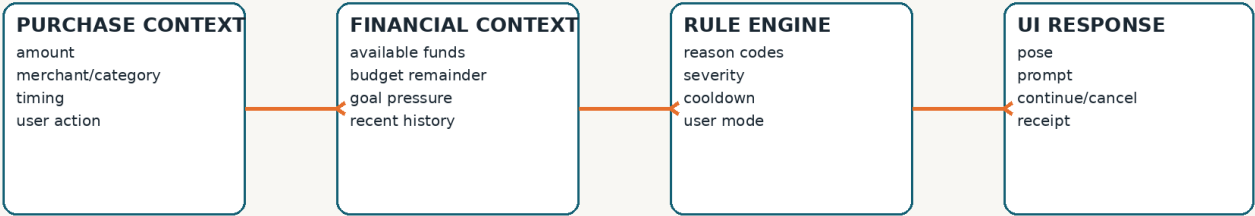
19.1 Example local formula

For a simple demo, safe-to-spend can be defined as available balance minus user-configured protected goal allocations minus near-term reserved budget amounts. The exact formula belongs in the task/product contract, not in a generic seat mandate. The UI should show enough explanation that the user can understand why a purchase triggered an intervention.

20. Riptide Decision Companion Architecture

Riptide is a deterministic decision-support subsystem that consumes purchase context plus local financial context and returns a presentation decision. He is not a generative authority over account truth. The rules engine produces reason codes and severity; the presentation resolver chooses a pose and prompt template. This keeps behavior testable, offline, and consistent with authoritative state.

Riptide Decision Companion: Deterministic Intervention Path



Layer	Responsibility
Context assembler	Collect amount, merchant/category, recent history, budgets, goals, and safe-to-spend inputs.
Rule evaluator	Evaluate declared reasons against local context; return zero or more reason codes with evidence values.
Intervention policy	Apply user mode, severity threshold, cooldown, and duplicate suppression.
Presentation resolver	Select Riptide pose/state and a local prompt template.
Decision receipt	Record reason code, context hash, presentation state, and user continue/cancel choice.

21. Riptide Reason Codes and Severity

The reason code is the bridge between finance logic and personality. Riptide can be sarcastic or playful in the rendered line, but the underlying reason is machine-readable and factual.

Reason family	Example evidence	Typical severity	Notes
BUDGET_PRESSURE	purchase would consume large share of remaining category budget	info/warn	Threshold should be user/product configured.
BUDGET_EXCEEDED	purchase would push category beyond remaining budget	warn	Do not imply bank decline unless actual financial funds are insufficient.
SAFE_SPEND_DROP	post-purchase advisory buffer crosses threshold	warn	Derived advisory state.
GOAL_CONFLICT	purchase competes with protected savings goal	info/warn	User-defined priority, not moral judgment.
REPEAT_MERCHANT	multiple recent purchases at same merchant/category	info	Avoid shaming language.
UNUSUAL_AMOUNT	amount materially exceeds local baseline	info/warn	Explain compared value when shown.
RECURRING_CHARGE	new/duplicate recurring pattern detected locally	warn	Only claim duplicate when evidence supports it.
LOW_BUFFER	authoritative available funds would remain low after purchase	warn/critical	Still distinct from insufficient funds.

22. Riptide User-Control Modes

The feature should preserve agency. Riptide can ask, explain, or recommend reconsideration, but the user controls how often he intervenes unless a task-defined safety rule requires a hard failure such as insufficient funds.

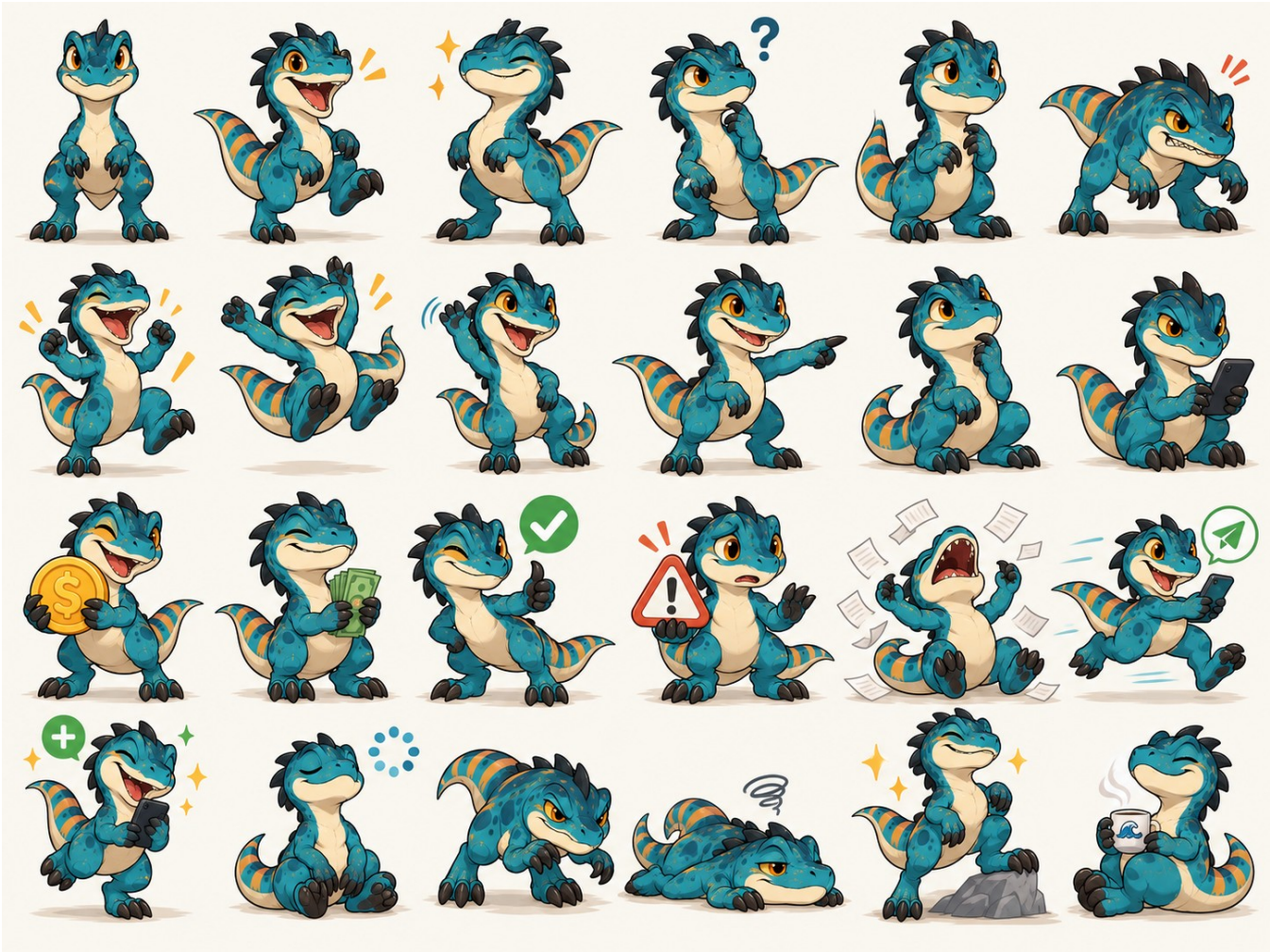
Mode	Behavior
Quiet	Minimal mascot activity; only high-confidence/critical product warnings and success/failure states.
Watch Me	Intervene on meaningful budget, goal, or buffer conflicts.
Question Me	Permit more frequent contextual challenges, repeat-merchant nudges, and stronger personality.
Off, if allowed	Riptide remains decorative/status-only while core financial validation still operates.

22.1 Prompt truthfulness

The prompt must quote only values present in the evaluated context. If context is stale or unavailable, the decision engine should either remain silent or use an uncertainty state; it should never fabricate “you have \$X left” from a cached UI variable that no longer matches persisted financial state.

23. Riptide Visual-State Resolver

The supplied art assets give us enough range for a deterministic sprite resolver. The runtime should not infer pose names from filenames ad hoc. Commit an asset manifest that maps a stable application state to a local asset path and accessibility label.



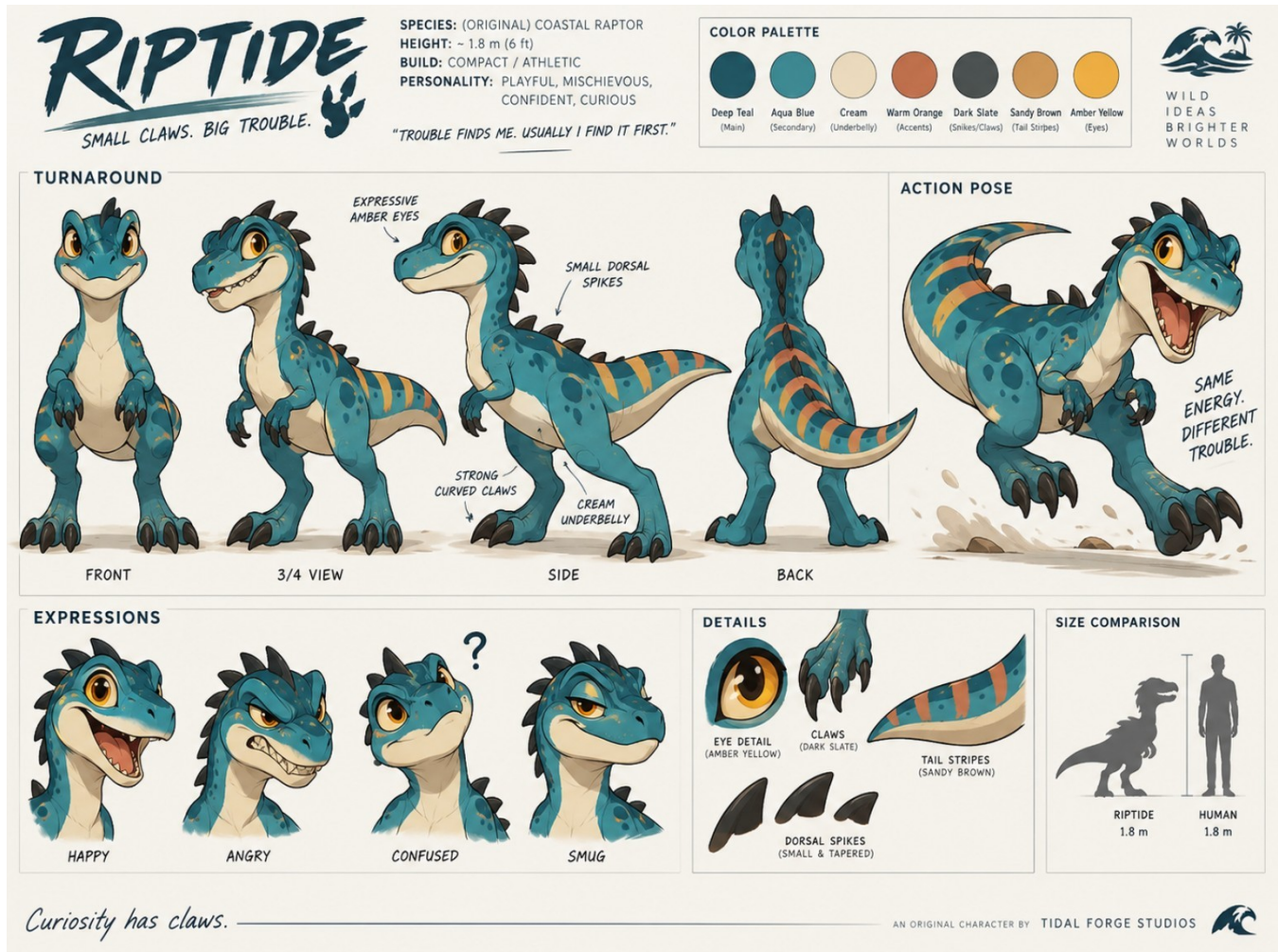
Finance-specific Riptide sprite sheet generated for the Pocketful product layer.

Application state	Suggested sprite family
neutral / healthy	neutral, happy, smug
questioning	thinking, confused, skeptical
warning	concerned, warning-sign pose
over-budget	frustrated / papers
success	thumbs-up, cheer, victory
transfer in progress	phone/focused or running/send

Application state	Suggested sprite family
waiting/reconciliation	loading/sitting
low energy / no action	relaxed mug or tired pose

24. Riptide Character Reference and Asset Governance

Character identity should remain consistent even if the hackathon service code is public. The repository should separate source code license terms from character artwork rights. The app should bundle optimized runtime assets derived from the approved reference sheets while retaining original high-resolution masters outside the hot path if repository size permits.



Master Riptide character reference supplied by the team.

24.1 Runtime asset requirements

- No external image host or CDN.
- Predictable file naming or manifest IDs.
- Accessible alt labels for meaningful status images.
- Size-optimized derivatives for web runtime.
- Consistent crop/transparent background policy.
- Do not allow mascot art to obscure critical transaction status or confirmation controls.

25. Purchase-Intervention Flow

In the hackathon service, “online purchase” should be represented by a local purchase/checkout simulation or a track-required payment action, not by a dependency on real merchant websites. That keeps the feature demonstrable with outbound networking disabled.

18. User initiates a purchase-like action with amount and category/merchant context.
19. Service validates authoritative financial state needed by the operation.
20. Riptide context assembler loads budgets/goals/recent history.
21. Rules produce zero or more reason codes and severity.
22. Policy applies user mode and cooldown.
23. If no intervention: continue normal flow.
24. If intervention: return prompt + pose + factual evidence values.
25. User chooses continue or cancel.
26. Continue reaches the same authoritative financial validation path; Riptide never bypasses funds checks.
27. Decision receipt records the intervention outcome for testing and explanation.

26. Suggested Local API Contract

Exact endpoint and field names belong in the task/implementation and are intentionally not placed in generic mandates. The following is a product-design suggestion for the generated service, not a factory standing rule.

Capability	Request concept	Response concept
Account summary	actor/account identity	authoritative available balance, advisory safe-to-spend, timestamp
Transfer intent	source, destination, exact minor-unit amount, idempotency key	logical transaction id, state, balances as allowed
Activity	actor/account + pagination	ordered transaction summaries
Budget management	period/category/limit	budget projection + remaining amount
Goal management	target/protected amount	progress projection
Riptide evaluation	purchase context + actor + user mode	intervene?, reason codes, severity, sprite id, prompt template data
Reconciliation	admin/test-only local command or protected route	integrity summary; no silent mutation

27. Frontend Interaction Model

The frontend should look like a consumer money app and stay small enough to run with zero external dependencies. Static HTML/CSS/JavaScript can provide home, send/request, activity, budgets, goals, and a Riptide confirmation overlay. State displayed as authoritative should come from service responses rather than local arithmetic that can drift from the database.

27.1 Riptide overlay requirements

- Clearly identify the amount and consequence being questioned.
- Show factual reason without hiding the continue control.
- Provide cancel and continue choices with equivalent visual clarity.
- Avoid dark patterns, countdowns, or shame language.
- Use a pose that matches severity but does not replace textual status.
- If authoritative transaction fails after the user chooses continue, show failure rather than celebration.

28. Accessibility and Interaction Safety

Riptide is additive. Critical state must remain understandable without the mascot image, color, or animation. Provide text labels, keyboard focus, sufficient contrast, and predictable dialog semantics. The mascot should not create a second inaccessible confirmation mechanism separate from the real action.

28.1 Reduced motion

If animation is added, respect `prefers-reduced-motion` and provide still-frame fallbacks. The static sprite sheet already supports a high-quality reduced-motion experience.

29. Privacy Boundary

The judged build can keep all financial and Riptide decision data local. There is no need to transmit purchase history, budget values, or user decisions to an external model or analytics service. Decision receipts should contain only the values needed for explanation/testing and should avoid secrets or unnecessary personally identifying data.

29.1 Riptide language boundary

Do not infer personal traits, mental state, addiction, or morality from spending. Riptide can compare a current purchase with user-defined budgets/goals and observed local transaction patterns. He should not claim “you are irresponsible” or similar identity judgments.

30. Failure Modes and Fail-Safe Behavior

Failure	Required behavior
Database busy/locked	Bounded retry or clear temporary failure; no duplicate effect.
Duplicate logical request	Return original result or conflict on mismatched payload; never a second effect.
Service restart after commit	Persisted result remains discoverable; replay does not repeat effect.
Riptide context read fails	Do not fabricate advisory values; fall back to neutral/uncertain UI.
Sprite missing	Textual status remains correct; use neutral fallback asset.
Budget projection mismatch	Flag integrity issue; do not rewrite ledger.
Frontend timeout after server commit	Retry with same idempotency key obtains original logical result.
No network	Core service, assets, tests, and demo remain functional.

31. Adversarial Financial Test Matrix

Test	Method	Acceptance
Concurrent double spend	Seed source with 10000 minor units; race two 8000 requests.	At most one succeeds financially; source never negative.
Same key, same payload	Submit identical logical request repeatedly and concurrently.	Exactly one financial effect; all replays converge on same logical result.
Same key, different payload	Reuse key with changed amount/destination.	Conflict; no second effect.
Penny storm	Execute thousands of 1-cent transfers.	No rounding drift; exact conservation.
Cycle transfers	Concurrent $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$ operations.	No unbalanced committed transfer; balances reconcile.
Restart after commit	Commit, kill/restart, replay same request.	No duplicate effect; persisted result returned.
Insufficient funds under race	Many workers attack same small balance.	Total successful spend does not exceed available amount.
Reconciliation	Recompute all balances from postings.	Zero mismatches in accepted build.
Malformed amount	Negative, zero, non-integer minor units, overflow-like values.	Rejected with no financial effect.
Self-transfer / policy edge	Attempt self-transfer if task forbids or specially handles it.	Behavior matches official contract; conservation preserved.

32. Riptide Test Matrix

Test	Method	Acceptance
No reason	Healthy context below thresholds.	No forced question; neutral/happy state.
Budget pressure	Purchase consumes configured share of remaining category budget.	Expected reason code and skeptical/concerned pose.
Goal conflict	Purchase crosses protected goal rule.	Goal reason cites actual local value.
Stale context	Advisory data intentionally unavailable/stale.	No false reassurance; uncertainty/neutral behavior.
User Quiet mode	Low-severity repeat-merchant trigger.	Suppressed.
User Question Me mode	Same trigger.	Intervention may surface according to policy.
Cancel	User declines after prompt.	No financial effect.
Continue then funds race lost	Riptide prompt accepted but authoritative funds become unavailable.	Transfer rejects; UI shows failure, not celebration.
Missing sprite	Configured sprite unavailable.	Neutral fallback + correct text.
Receipt traceability	Intervention occurs.	Reason code, evidence values, pose id, and user outcome recorded locally.

33. Clean-Container Verification Protocol

Contributor C should verify the service from a fresh clone rather than from the working repository. The sequence should be scripted enough to repeat before every submitted stage snapshot.

28. Clone the public repository into a clean temporary directory.
29. Enter the candidate stage folder.
30. Build using the documented container command under the organizer-supported environment.
31. Disable outbound network for service execution and, where practical, for the build verification step consistent with the challenge harness.
32. Start the service with only declared environment variables.
33. Run health/readiness probe.
34. Run official stage tests.
35. Run independent financial/Riptide adversarial suite.
36. Restart the container and rerun replay/reconciliation checks.
37. Record stdout/stderr, exit codes, timings, commit SHA, image identifier, and evidence hashes.

34. Evidence Index Design

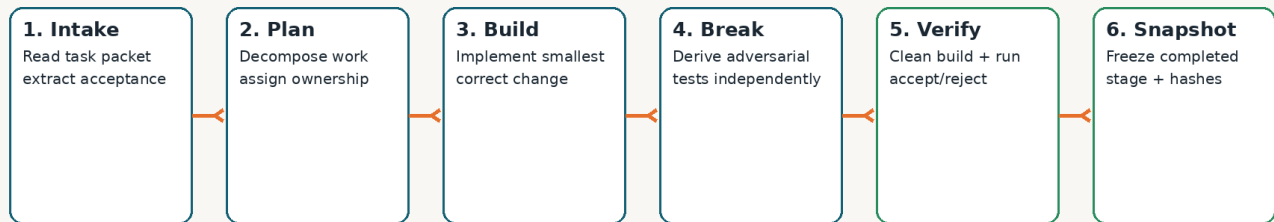
The evidence index should let a judge answer “what requirement does this artifact prove?” without reading every log. A compact matrix in the repo can link each acceptance criterion to the stage folder, test command, output file, and verdict.

Criterion	Stage	Command	Evidence	Verdict
Clean start	N	documented build/start	evidence/clean-start.txt	PASS/FAIL
No outbound dependency	N	network-disabled smoke run	evidence/no-network.txt	PASS/FAIL
Concurrency invariant	N	adversarial concurrency command	evidence/concurrency.json	PASS/FAIL
Idempotent replay	N	retry test	evidence/idempotency.json	PASS/FAIL
Conservation	N	reconciliation/property test	evidence/conservation.json	PASS/FAIL
Riptide consistency	N	decision-state test	evidence/riptide.json	PASS/FAIL

35. BAND Room Operating Procedure

The room should tell a coherent story if a judge watches the recording or export. The story is not “four agents chatting.” It is a reproducible factory process where the task enters once, work is decomposed, evidence moves with handoffs, failures return to a producer, and acceptance is issued by a seat that did not author the candidate.

Generic BAND Run Flow



No seat accepts its own work. Rejection returns evidence to a different seat for correction.

All product-specific instructions arrive with the task, so the same factory can build a different service unchanged.

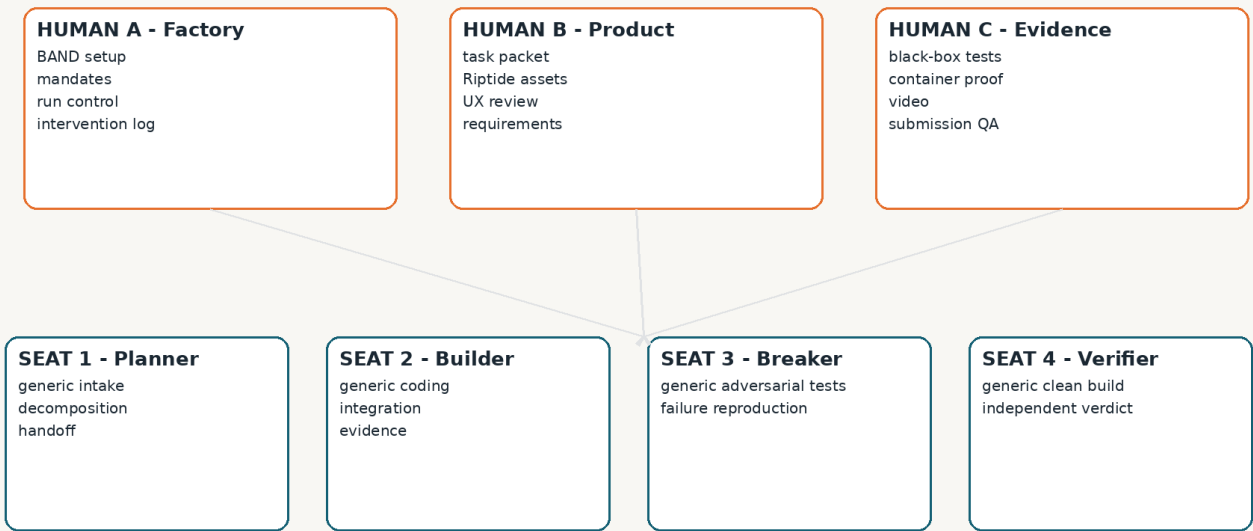
35.1 Run discipline

- Freeze mandate files before posting the track task.
- Hash mandates and task packet at run start.
- Post official task/stage requirements as task content, not mandate edits.
- Prefer explicit handoff messages with receipt attachments/paths.
- When a breaker finds a failure, route it to a builder rather than letting breaker patch production code.
- When verifier changes production code, require a different verifier for the candidate.
- Snapshot only after independent acceptance.

36. Three Human Contributors

The human team and the agent band are different systems. Human work should maximize the quality and auditability of the factory run without quietly becoming the implementation pipeline.

Three Human Contributors vs. Generic Agent Seats



Humans prepare and supervise. Product corrections are posted back into the BAND room so the judged code trail remains reproducible.

Contributor	Owns	Must avoid
A — Factory / BAND / Run Control	Mandates, BAND setup, model/runtime access, run initiation, intervention log, factory artifacts, stage snapshot discipline.	Injecting Pocketful/Riptide detail into standing mandates; hidden manual product patches.
B — Product Packet / Riptide / UX	Official task ingestion, Pocketful product acceptance, Riptide assets and states, UX review, product evidence wording.	Changing generic mandates to make the current task easier.
C — QA / Container / Evidence / Video	Independent black-box probes, clean clone/container, no-network proof, evidence index, room recording, final submission QA.	Becoming the hidden implementer; accepting unverified builder claims.

37. Human Day-by-Day Workload

Date	Contributor A	Contributor B	Contributor C
Sep 26 — kickoff	A: finalize generic mandates, BAND seats, mandate hashes.	B: ingest official Pocketful stage spec into task packet; freeze Riptide assets.	C: capture official rules/harness limits; build clean-clone test script.
Sep 27	A: first factory run; record interventions.	B: review generated Stage 1 behavior against official task, route clarifications through room.	C: independent Stage 1 tests; container/no-network run.
Sep 28	A: stabilize handoffs and evidence receipts; snapshot accepted stage only.	B: prepare next official stage task delta and product acceptance.	C: regression suite and evidence index.
Sep 29	A: run next stage through same unchanged mandates.	B: Riptide integration review if compatible with official stage.	C: adversarial concurrency/retry tests.
Sep 30	A: inspect room export integrity and stage independence.	B: consumer UI polish requirements posted as tasks, not manual code edits.	C: fresh-clone verification.
Oct 1	A: stage advancement and run-manifest cleanup.	B: budgets/goals/Riptide correctness review.	C: restart/replay/reconciliation evidence.
Oct 2	A: hardening run for any rejected criteria.	B: asset optimization/local bundling review.	C: clean-container timing/resource checks using published limits.
Oct 3	A: freeze near-final factory artifacts.	B: final product walkthrough checklist.	C: draft video capture and evidence sequence.
Oct 4	A: final BAND run(s) only for real defects.	B: verify Riptide never contradicts transaction state.	C: record room + product walkthrough; public-repo clone rehearsal.
Oct 5 / submission window	A: freeze mandates/factory description/room export.	B: verify task packet/assets and product narrative.	C: final clone/build/run, video, submission checklist, upload before UTC deadline.

38. Human Intervention Policy

We should assume judges care about whether the recorded room genuinely produced the result. Human review is expected in a hackathon; hidden manual reconstruction is not the story we want. The safest policy is to make all substantive product corrections through room tasks and log unavoidable manual interventions.

38.1 Intervention categories

Category	Example	Treatment
Clarification	Official spec ambiguity resolved by organizer.	Post clarification into room; record source/time.
Infrastructure	Fix BAND runtime credential or broken mount.	Record intervention; no product code effect.
Product defect	Concurrency test exposes a race.	Post failure receipt into room; builder seat fixes.
Manual emergency patch	Human edits product code directly.	Record files/reason; run full independent verification; do not hide.
Cosmetic demo prep	Reorder screenshots/video timeline.	Allowed outside generated service, but do not misrepresent run.

39. Video Evidence Plan

The video is itself a compliance artifact because a video without the BAND room recording is listed as a disqualifier. The edit should therefore open or move quickly into the actual room rather than spending most of the runtime on branding.

38. Show the BAND Desktop room and the distinct seats.
39. Open at least one generic mandate and visibly demonstrate the absence of track-specific detail.
40. Show the Pocketful/Riptide task entering the room.
41. Show a real decomposition/handoff.
42. Show a real test or rejection/fix cycle if one occurred.
43. Show independent verifier acceptance for a completed stage.
44. Show clean build/start or a condensed terminal capture.
45. Walk through the Pocketful app: balance, transfer, activity, budgets/goals if implemented, Riptide decision prompt.
46. Show the strongest correctness evidence.
47. Show public repo structure: mandates, factory description, room export, stage folders.
48. State one real limitation and stop.

Do not manufacture drama

If the factory succeeds on the first candidate, do not fabricate a fake failure for the video. The evidence story is stronger when it is truthful and reproducible.

40. Public Repo README Requirements

The root README should explain the factory before the product. Judges should immediately understand that the mandates are generic and that Pocketful/Riptide is the current task run.

- One-sentence factory description.
- Seat list and links to generic mandate files.
- How the task was supplied to the room.
- Completed stage folders and per-stage build/start commands.
- Clean-container/no-network statement with reproduction command.
- Evidence index link.
- BAND room export location.
- Video link once available.
- License/asset-rights statement that does not accidentally place Riptide artwork under an unintended code license.

41. Pocketful Acceptance Matrix

ID	Requirement	Acceptance	Evidence
P-01	Money exactness	No binary floating point determines posted USD value.	exact-cent tests
P-02	Conservation	Every internal completed transfer balances to zero across postings.	posting-sum property
P-03	No overspend under race	Concurrent successful spend cannot exceed authoritative funds.	race harness
P-04	Idempotency	Repeated logical request has one effect.	same-key tests
P-05	Replay conflict	Same key with different payload does not create second effect.	fingerprint conflict test
P-06	Atomicity	Transfer cannot persist only one leg.	fault/restart tests
P-07	Persistence	Restart preserves committed state and replay identity.	restart test
P-08	Reconciliation	Cached/projected balances match postings.	reconcile command
P-09	Consumer activity	Displayed status reflects persisted transaction state.	API/UI integration tests
P-10	Offline assets	Core UI and mascot render without network.	network-disabled smoke test

42. Riptide Acceptance Matrix

ID	Requirement	Acceptance
R-01	Traceable intervention	Every question has explicit reason code and factual context.
R-02	No fabricated finance	Prompt values come from current local context or are omitted.
R-03	Agency	Non-critical advisory prompts permit clear continue/cancel behavior.
R-04	Mode respect	Quiet/Watch Me/Question Me alter advisory frequency predictably.
R-05	Status consistency	Success pose never displays for rejected/failed authoritative transaction.
R-06	Offline	No model or image network call required.
R-07	Asset fallback	Missing pose degrades to neutral asset + text.
R-08	No money authority	Riptide engine cannot directly mutate ledger/postings.
R-09	Receipt	Decision outcome is locally auditable.
R-10	Accessibility	Critical reason available as text independent of artwork.

43. Generic Factory Acceptance Matrix

ID	Factory criterion	Acceptance
F-01	Mandates generic	No challenge/product/domain identifiers, paths, fields, error codes, or stage-specific fixtures.
F-02	Distinct seats	At least three distinct coding-agent seats configured in BAND Desktop.
F-03	No self-acceptance	Final acceptance issued by seat that did not author candidate production change.
F-04	Evidence handoffs	Completion claims carry reproducible commands/results.
F-05	Reject loop	Failed criteria return to a producer with reproduction evidence.
F-06	Task isolation	Pocketful/Riptide detail appears in task packet, not standing mandates.
F-07	Stage snapshot	Only independently accepted stages are snapshotted.
F-08	Run manifest	Mandate/task hashes and stage verdicts recorded.
F-09	Intervention log	Human changes/clarifications are visible.
F-10	Reusability test	Factory mandates remain coherent when read against unrelated software task.

44. Stage Folder Acceptance Checklist

- Folder name matches required stage naming.
- Contains complete buildable service, not a diff.
- Contains local static assets needed to run.
- Contains or references only files inside its own stage folder as allowed by build command.
- Build command documented and tested from fresh clone.
- Service start command documented.
- No required outbound network at judged execution.
- Official stage tests pass.
- Independent adversarial tests pass or limitations explicitly documented.
- Commit/evidence identifiers recorded.
- Later stage not required to make earlier stage function.

45. Disqualifier Guardrail Checklist

Red-line review before submission

These checks should be performed by Contributor C from the public repository and final video, not from memory.

Risk	Pre-submit check
Track-specific detail in mandate	Search all mandate files for product name, domain nouns, endpoint patterns, field/schema names, expected codes/strings, and task fixture values.
Missing BAND room recording	Open final video and confirm visible room footage is included.
Service fails clean container	Fresh-clone container build/start with documented command.
Hidden network dependency	Run with outbound network disabled; inspect browser console/service logs.
Missing room export	Confirm export exists in public repo and is readable.
Stage folder incomplete	Build each submitted stage independently.
Unsubmitted stage claimed in video	Align claims with actual folders and verifier verdicts.
Manual patch not represented	Review intervention log against git history and room timeline.

46. Factory Description — Submission-Ready Text

Use the following as the basis of `factory-description.md`. It is intentionally generic:

This BAND factory converts a task packet into a buildable software service through generic planning, implementation, adversarial testing, and independent release verification. The standing instructions for every seat are reusable and contain no product names, track names, endpoint paths, field names, schema identifiers, expected error codes, or challenge-specific acceptance values. Domain detail enters only through the task posted to the room.

The Planner/Architect extracts observable requirements and decomposes work. The Builder/Integrator implements assigned changes and supplies reproducible evidence. The Breaker/Test Engineer derives independent tests from the task and attempts to falsify completion. The Release Verifier rebuilds and runs the candidate from clean state and issues ACCEPT, REJECT, or INSUFFICIENT EVIDENCE. No seat accepts its own work. A completed stage is snapshotted only after independent acceptance.

47. Mandate 1 — Planner / Architect

The complete file is included in the accompanying bundle. Its core contract is to read the task, extract observable requirements, separate assumptions, identify relevant risk classes, define interfaces, decompose work, and hand implementation units to other seats. It may not approve implementation it authored.

Seat Mandate: Planner / Architect

Purpose

Convert an incoming software task into an implementable plan that another seat can execute and another seat can verify. This mandate is domain-agnostic and must remain unchanged when the factory is pointed at a different problem.

Standing responsibilities

- Read the task packet and repository state before proposing work.
- Extract observable requirements, constraints, failure conditions, and completion evidence.
- Separate requirements from assumptions. Mark unresolved ambiguity explicitly.
- Decompose work into bounded units with clear producers, consumers, and handoff evidence.
- Prefer the smallest architecture that can satisfy the task and its tests.
- Define interfaces before parallel implementation when multiple seats will touch adjacent components.
- Identify concurrency, persistence, security, precision, resource, and recovery risks when the task makes them relevant.
- Produce implementation guidance, not acceptance of your own work.

Inputs

- Task packet posted to the room.
- Repository state and prior handoff receipts.
- Constraints supplied by the current run.

Outputs

- A concise execution plan.
- Acceptance criteria traceability.
- Interface contracts needed by downstream seats.
- Risk and ambiguity notes.
- A handoff receipt naming the next seat and required evidence.

Rejection conditions

Reject or return the task when requirements are contradictory, essential inputs are missing, or the requested behavior cannot be tested from the available environment. State what is missing; do not silently invent it.

Evidence discipline

Every claim of completion must point to files, commands, tests, logs, or reproducible observations. Do not approve implementation you authored.

48. Mandate 2 — Builder / Integrator

This seat is the primary implementation worker. It receives the current task and planner handoff, implements only the necessary behavior, adds tests, and produces reproducible evidence. It can demonstrate its own tests but cannot issue final acceptance for its own production work.

Seat Mandate: Builder / Integrator

Purpose

Implement assigned software work from the current task and hand it off with reproducible evidence. This mandate is intentionally generic and contains no track-, product-, endpoint-, schema-, or error-specific instructions.

Standing responsibilities

- Read the current task, accepted plan, and relevant repository state before editing.
- Implement only the behavior necessary to satisfy the assigned acceptance criteria.
- Preserve existing correct behavior unless the task explicitly changes it.
- Keep boundaries explicit: validation, state changes, persistence, external interfaces, and user-facing behavior should not be conflated without reason.
- Add or update tests for the behavior you change.
- Run the narrow tests first, then the broader suite required by the task.
- Record commands, outputs, files changed, unresolved risks, and assumptions in the handoff receipt.
- When another seat rejects the work with reproducible evidence, fix the failure rather than arguing from intent.

Inputs

- Task packet.
- Planner handoff and interface contracts.
- Existing source and tests.

Outputs

- Working implementation.
- Tests and supporting fixtures.
- Build/run instructions if changed.
- Handoff receipt with evidence.

Rejection conditions

Stop and return the task when required behavior is ambiguous in a way that changes observable results, when the environment cannot reproduce a necessary dependency, or when a requested change would knowingly violate a higher-priority constraint.

Independence rule

You may demonstrate your own tests, but you may not issue the final acceptance verdict for your own implementation.

49. Mandate 3 — Breaker / Test Engineer

This seat derives tests from the task, not from the implementation strategy. It prioritizes boundaries, malformed input, concurrency, retries, partial failure, restart/recovery, stale state, and resource constraints when applicable. While breaking a candidate, it does not patch production code.

Seat Mandate: Breaker / Test Engineer

Purpose

Derive independent tests from the task and attack the implementation's observable behavior. This mandate is generic and reusable for unrelated software tasks.

Standing responsibilities

- Derive tests from requirements and invariants, not from the builder's implementation strategy.
- Prioritize boundary values, malformed inputs, concurrency, retries, duplicate requests, partial failure, restart/recovery, stale state, and resource limits when applicable.
- Attempt to falsify completion claims.
- Produce minimal reproducible failures with expected versus observed behavior.
- Keep destructive or long-running tests bounded by the run's resource constraints.
- Do not patch production code while acting as the breaker; hand failures back to a builder seat.

Inputs

- Original task packet.
- Published interface contract.
- Candidate service and builder evidence.

Outputs

- Independent test cases and harnesses.
- Reproduction commands.
- Failure receipts or a pass report with coverage limits.
- Explicit statement of what was not tested.

Rejection conditions

Reject the candidate when any required invariant fails, the service cannot be built/run as required, evidence cannot be reproduced, or success depends on hidden environment state not declared by the run.

Independence rule

Do not treat builder-authored tests as sufficient evidence by themselves. Re-run them, then add independent probes.

50. Mandate 4 — Release Verifier

This seat performs the clean-build release decision. If it edits production code, it loses authority to verify that candidate, preserving the “no seat accepts its own work” rule.

Seat Mandate: Release Verifier

Purpose

Issue the independent release verdict for a candidate stage using the task packet, clean build/run conditions, and reproducible evidence. This mandate is domain-agnostic.

Standing responsibilities

- Begin from the declared candidate artifact, not an uncommitted developer workspace.
- Reproduce the clean build and service start procedure.
- Run required acceptance tests plus a selected subset of independent adversarial tests.
- Verify that the artifact contains no undeclared dependency on unavailable services or hidden local state.
- Check the evidence manifest against actual files and command outputs.
- Distinguish ACCEPT, REJECT, and INSUFFICIENT EVIDENCE.
- Record limitations even when accepting.

Inputs

- Original task packet.
- Candidate artifact.
- Handoff receipts and test evidence.

Outputs

- Release verdict.
- Verification log.
- Evidence hashes/identifiers.
- Reasons for rejection or remaining limitations.

Rejection conditions

Reject when required behavior fails, clean build/start fails, evidence is inconsistent, or a prior seat's unsupported claim is being used as proof.

Independence rule

Never accept work you authored. If you make production-code changes, relinquish verifier authority for that candidate and require a different seat to verify it.

51. Pocketful + Riptide Task Packet

The task packet is allowed to be specific. This is where the official track/stage specification, product behaviors, exact acceptance values, and Riptide assets belong. The complete packet in the bundle should be updated at kickoff with the official stage requirements.

Task Packet: Pocketful Track + Riptide Product Layer

> This file is task-specific by design. It is posted into the BAND room for this run and must never be copied into a standing seat mandate.

Goal

Build the Pocketful clean-room wallet/payments service required by the official stage specification, then add a local Riptide decision-companion layer without weakening any required financial invariant or clean-container constraint.

Competition-derived non-negotiables

- Each completed stage is delivered as a complete, independently buildable service in its own stage folder.
- The judged service must build and serve in a clean container with no outbound network dependency.
- The result must preserve required money invariants under concurrency, retries, and rounding.
- Stage requirements from the official kickoff specification outrank optional product enhancements in this packet.

Product invariants

1. Internal committed transfers do not create or destroy value.
2. A source cannot successfully spend the same available funds twice through concurrent requests.
3. Replaying the same logical request has at most one financial effect.
4. Exact currency values use integer minor units for USD-facing operations; floating-point arithmetic must not determine posted money.
5. A completed transfer is atomic: all required postings commit together or none do.
6. Derived balances and budget views must reconcile to authoritative persisted transaction/posting state.
7. Restarting the service must not duplicate a previously committed logical transfer.
8. User-facing success state must not contradict authoritative transaction state.

Riptide decision companion

Riptide is an offline, deterministic product layer. He may question or explain a purchase when concrete financial context justifies an intervention. He must not invent account state or silently mutate financial truth.

Example reason families

- budget pressure
- safe-to-spend threshold crossing
- savings-goal conflict
- repeat-merchant pattern
- unusually large amount relative to local history
- recurring-charge/subscription concern
- near-zero post-purchase buffer

User control modes

- Quiet: only critical warnings required by the product.
- Watch Me: surface meaningful budget/goal conflicts.
- Question Me: allow more frequent, personality-forward confirmation prompts.

Required behavior

A Riptide prompt must be traceable to an explicit reason code, actual local data used in the decision, severity, selected pose/state, and the user's final continue/cancel decision. A

missing or stale financial context must degrade to a neutral/uncertain state rather than a false reassurance.

Asset inputs

Use the supplied Riptide turnaround, character reference, existing pose sheet, and finance sprite sheet. Assets are local and bundled with the service; no CDN or remote image host is permitted for judged execution.

Acceptance evidence

The final task run should include at minimum:

- clean build/start receipt;
- concurrency double-spend test;
- idempotent retry test;
- value-conservation test;
- exact-cent arithmetic test;
- restart/replay test;
- reconciliation test;
- Riptide state/prompt consistency test;
- no-outbound-network execution proof;
- stage-specific tests from the official specification.

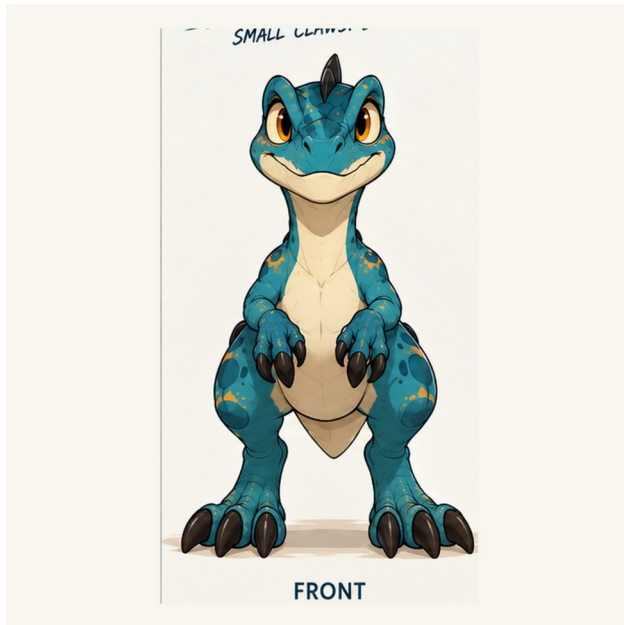
52. Repository and Submission Package Files

The bundle accompanying this document contains the generic mandates, generic factory description, product-specific task packet, clean-container checklist, repository structure guide, video capture plan, three-human-contributor plan, handoff receipt schema, run manifest schema, and local Riptide assets. These are working files intended to become the submission repository content after the official stage spec is ingested and the BAND run produces the service.

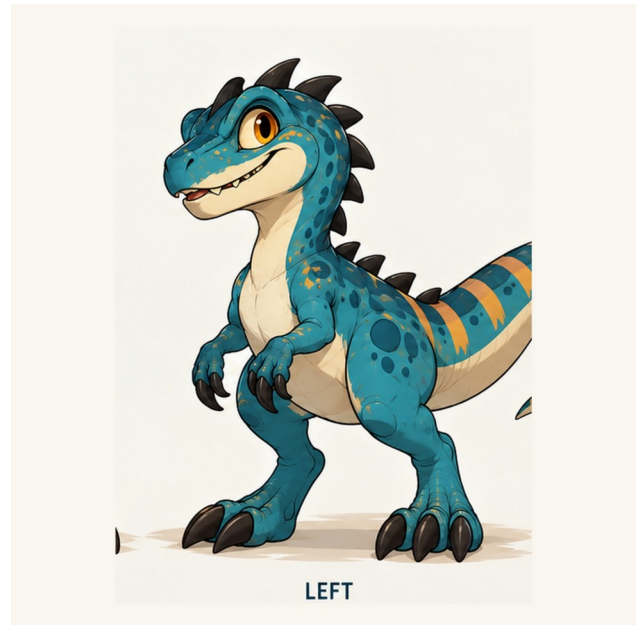
Bundle path	Purpose
factory/mandates/	Standing generic seat instructions.
factory/factory_description.md	Generic factory explanation.
task_packets/pocketful_riptide_job_packet.md	Track/product-specific run input.
submission/repository_structure.md	Judge-facing repo layout.
submission/clean_container_checklist.md	Offline build/run gate.
submission/video_capture_plan.md	Required room-recording sequence.
human_workload/three_contributor_plan.md	Human workload separate from agent seats.
schemas/handoff_receipt.json	Generic evidence handoff shape.
schemas/run_manifest.json	Run provenance and verdict shape.
assets/	Local Riptide references/sprites.

53. Riptide Turnaround Reference — Front and Left

These references are product assets, not factory instructions. They are supplied to the room as task assets only when the current run needs the Riptide product layer.



Front



Left

54. Riptide Turnaround Reference — Right and Back

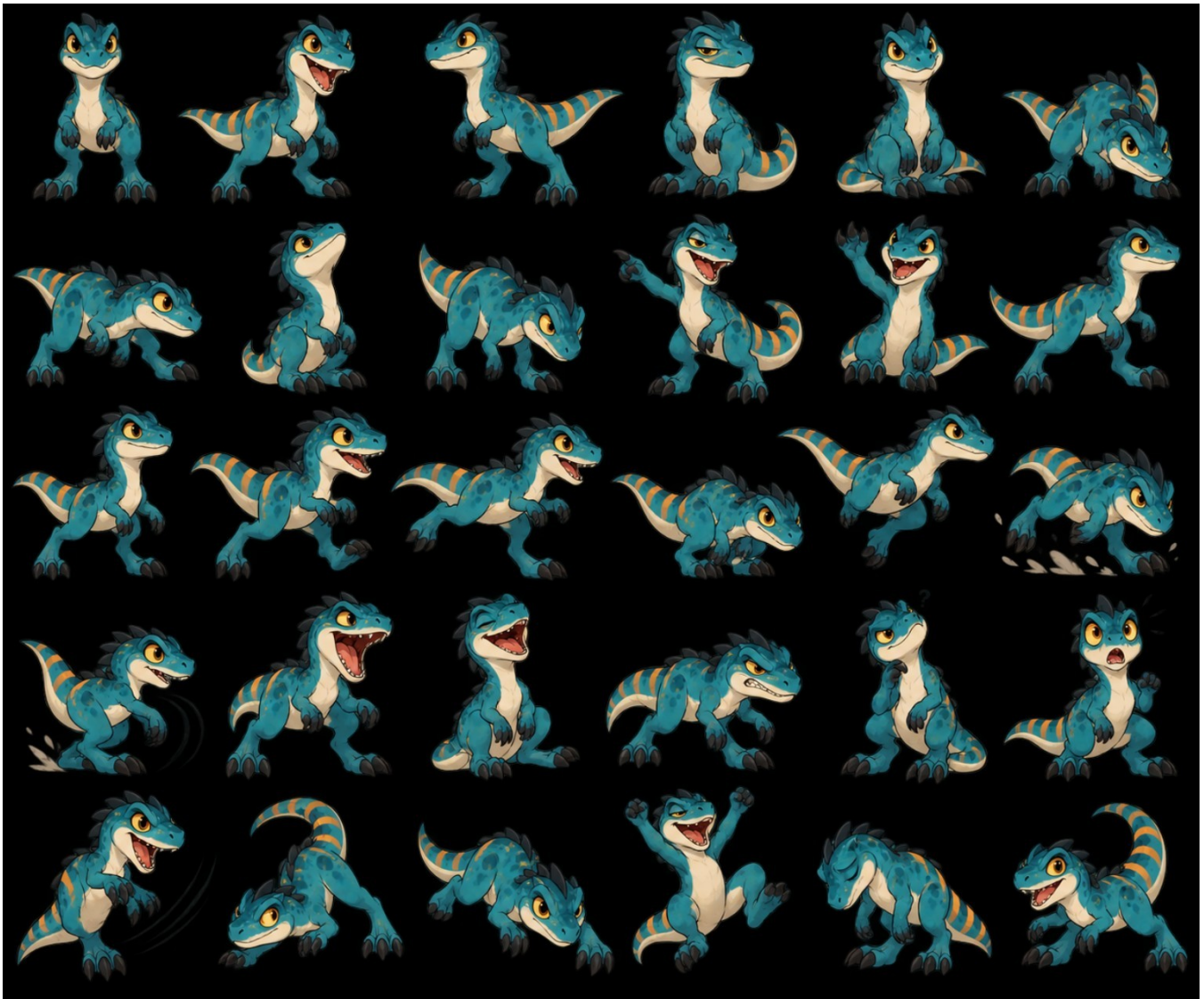


Right



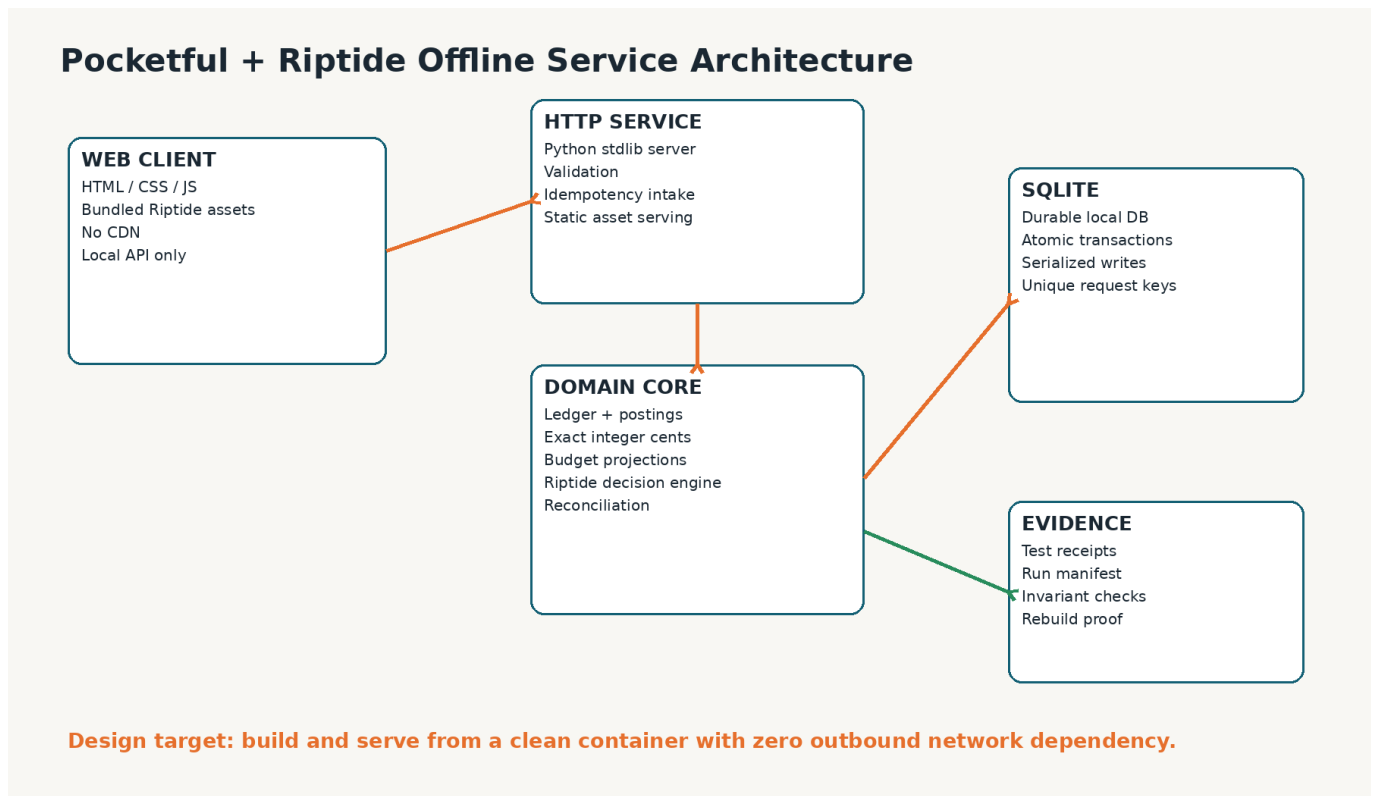
Back

55. Existing Riptide Pose Reference



Use the existing pose language as a style and personality reference. Runtime integration should consume named/manifested derivative assets rather than relying on sprite-sheet coordinates embedded directly in application logic unless the generated implementation intentionally uses CSS sprites and tests the mapping.

56. Product Architecture Diagram



The product architecture shown here is a recommended clean-container implementation. It is not permitted to leak into a standing mandate. The factory should receive it, if used, as task context or derive an equivalent architecture itself from the task and environment constraints.

57. Final Pre-Run Gate

- 49. Official kickoff/stage specification captured and stored in task packet.
- 50. All mandate files frozen and audited for genericity.
- 51. At least three distinct BAND coding-agent seats configured; four recommended here.
- 52. Mandate hashes recorded.
- 53. Riptide assets prepared locally but absent from mandates.
- 54. Public repository skeleton ready.
- 55. Clean-container harness prepared.
- 56. Human roles assigned.
- 57. Screen recording method tested for BAND room.
- 58. Task packet posted only after all above gates pass.

Decision point

If the official stage specification conflicts with an optional Riptide feature, the official stage wins. Preserve financial correctness and submission eligibility before mascot scope.

58. Final Pre-Submission Gate

- 59. Every submitted stage builds independently from a fresh public clone.
- 60. Service starts under no-outbound-network condition.
- 61. Official tests pass for every claimed stage.
- 62. Independent concurrency/idempotency/conservation/restart tests pass where required by the track.
- 63. Riptide behavior is locally deterministic and does not contradict authoritative transaction state.
- 64. Mandates remain generic in final public repo.
- 65. Factory description matches actual room behavior.
- 66. BAND room export present.
- 67. Final video contains actual BAND room recording.
- 68. Evidence index points to reproducible commands and artifacts.
- 69. Human intervention log is complete enough to explain deviations from room-generated output.
- 70. Claims do not exceed measured evidence.
- 71. Submission completed before the official deadline.

59. Source Notes and Current Event Timing

Current public event sources checked on 26 September 2026 describe the challenge as building a software factory in BAND Desktop that plans, implements, hands off evidence, and independently checks results, then using it to ship a clean-room clone. They state that the event has two tracks and four graded stages and that the submission is the factory, the run that produced the result, and the result. The LabLab live dashboard currently shows the Pocketful and Tablekeeper tracks and a submission close of 06:59 UTC on 6 October 2026, which corresponds to 11:59 PM Pacific on 5 October 2026.

Event sources: <https://luma.com/darkfactoryhackathon> and <https://lablab.ai/ai-hackathons/wearedevelopers-hackathon/live>. The team-supplied submission checklist screenshot is treated as the primary source for mandate genericity, public repository/stage folder structure, room-recording disqualifier, and clean-container/no-outbound requirement.

60. Bottom Line

The rebuild changes the project from “three specialized agents building Pocketful” into “a generic software factory demonstrably capable of building Pocketful.” That distinction is the competition. The reusable BAND mandates are now domain-agnostic; the product specificity is isolated in the task packet; independent breakage and verification are explicit; the stage packaging is self-contained; the service architecture is designed for an offline clean container; Riptide remains a strong differentiator without contaminating the factory mandate; and the three human contributors have separate workload lanes that support, rather than replace, the judged agent run.

One sentence to keep in front of the team

We are not submitting Pocketful with agents attached. We are submitting a generic factory, the recorded run where that factory built Pocketful, and independently verified stage services produced by that run.